Introducción a la Ciencia de Datos

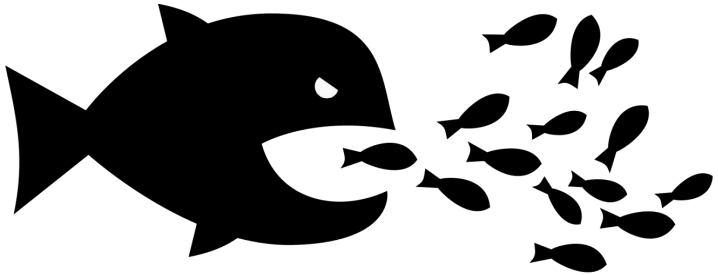
Maestría en Ciencias de la Computación

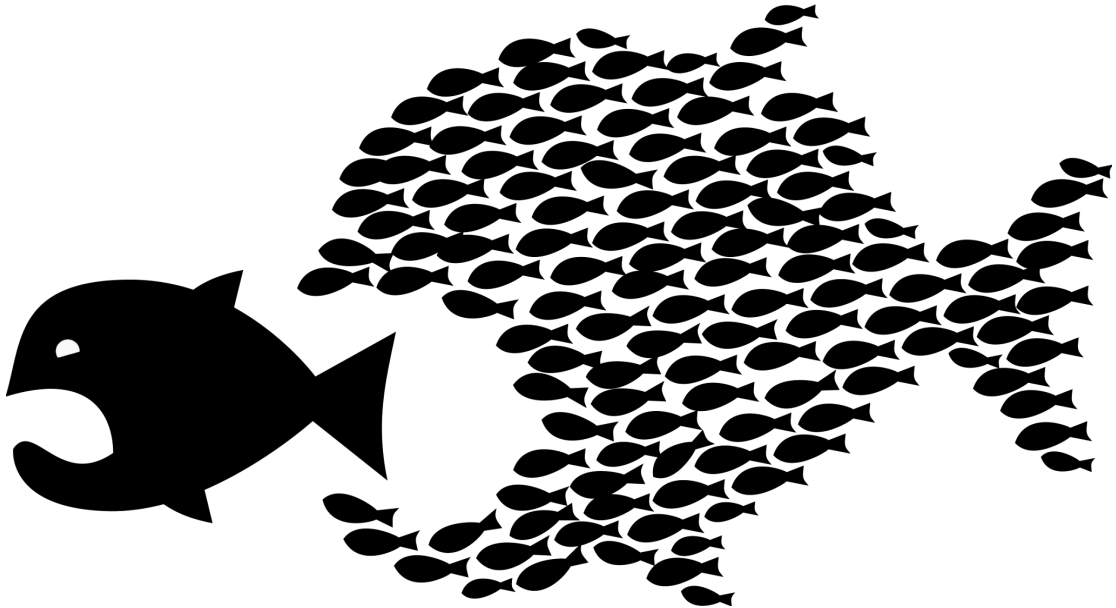
Dr. Irvin Hussein López Nava

Laboratorio de Ciencia de Datos y Aprendizaje Automático

05

Meta-aprendizaje





5.1 Ensambles


¿Qué es
el Aprendizaje
basado en Ensambles?

Definiciones

ChatGPT

Es una técnica en el campo del aprendizaje automático en la que se combinan múltiples modelos para mejorar el rendimiento y la precisión de las predicciones en comparación con un solo modelo. Se basa en la idea de que la combinación es más robusta y generalizable que un solo modelo.

https://chat.openai.com/

Wikipedia

En estadística y aprendizaje automático, los métodos de ensamble utilizan múltiples algoritmos de aprendizaje para obtener un rendimiento predictivo mejor que el que podría obtenerse con cualquiera de los algoritmos de aprendizaje constituyentes por sí solos.

https://en.wikipedia.org/wiki/Ensemble_learning

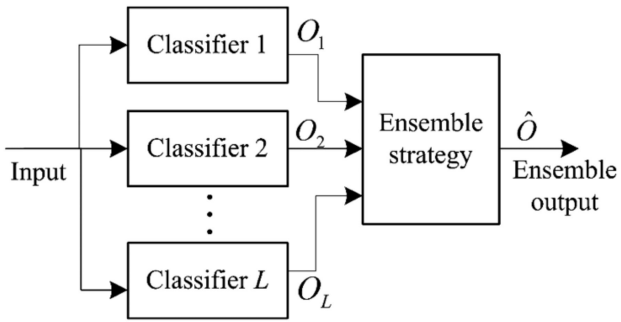
Gemini

Es un enfoque de aprendizaje automático que consiste en combinar los resultados de varios modelos de aprendizaje automático para mejorar el rendimiento general. Al combinar los resultados de los modelos, se puede reducir el sesgo y mejorar la robustez.

https://gemini.google.com/

¿Qué es?

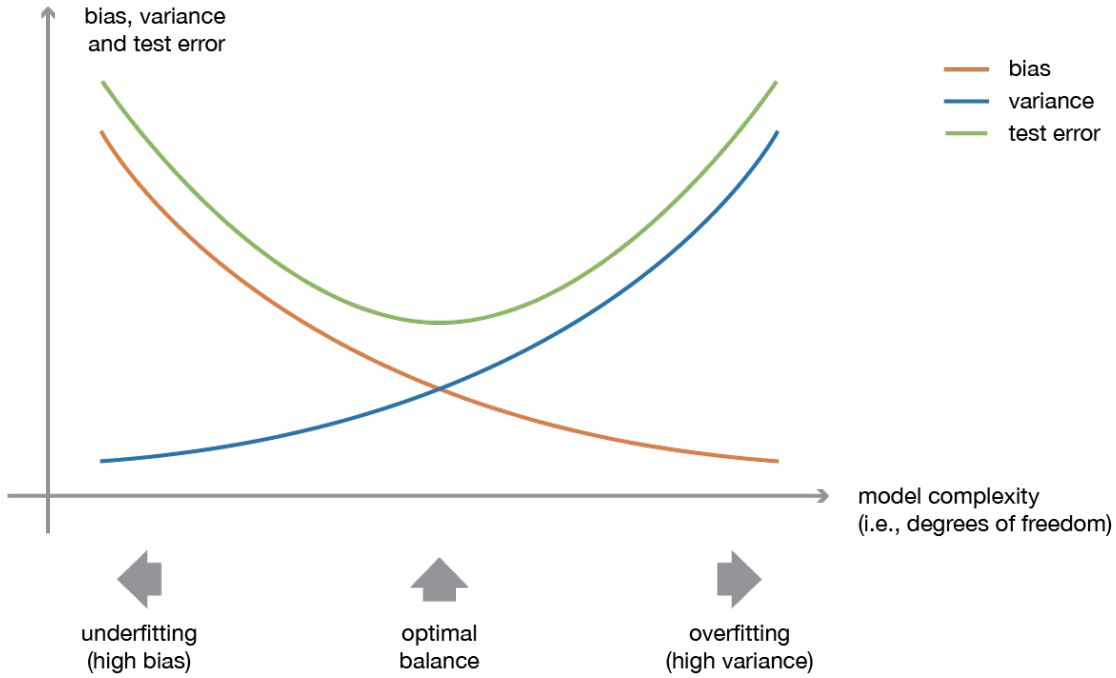
- Es un paradigma de aprendizaje automático que utiliza múltiples algoritmos de aprendizaje (a menudo denominados “*weak learners*” o "aprendices débiles") para obtener un rendimiento predictivo mejor que el que podría obtenerse con cualquiera de los constituyentes por sí solos.



Single weak learner

- **Un dato**: la elección del algoritmo de inferencia es extremadamente importante para tener alguna posibilidad de obtener buenos resultados.
- Esta elección puede depender de muchas variables: cantidad de datos, dimensionalidad del espacio, hipótesis de distribución...
- Un **sesgo** y una **varianza** bajos, aunque la mayoría de las veces varían en direcciones opuestas, son las dos características fundamentales que se esperan de un modelo.
- Se trata del conocido equilibrio (*tradeoff*) entre sesgo y varianza.

Bias-variance tradeoff



Weak learners

- Un **aprendiz débil**: con cualquier distribución de los datos, produce un clasificador apenas mejor que el azar:

$$\text{error} \leq \tfrac{1}{2} - \gamma, \qquad \gamma > 0$$

- Schapire (1990) demostró que combinar aprendices débiles da uno fuerte: es el origen del boosting (sesión 18).
- **Modelo base** es cualquier modelo que entra en un ensamble, débil o no:
  - **Boosting**: árboles de un nivel, con sesgo alto y varianza baja. **Son débiles**.
  - **Bagging**: árboles profundos, con sesgo bajo y varianza alta. **No son débiles**.
  - **Stacking**: modelos de distinto tipo, casi siempre buenos por sí solos.

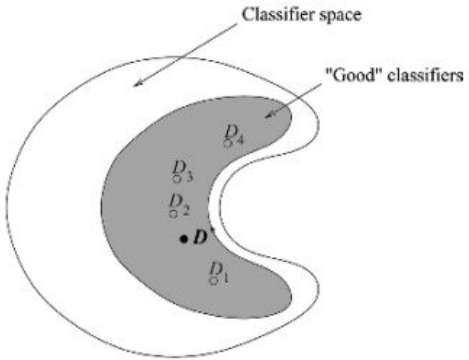
Ensemble theory

- Los algoritmos de aprendizaje buscan a través de un **espacio de hipótesis** para encontrar una hipótesis adecuada para un problema concreto; aunque exista, es **muy difícil encontrarla**.
- Los **ensambles** combinan múltiples hipótesis para formar una hipótesis (con suerte) mejor.
- Evaluar la predicción de un **ensamble** requiere más cálculo que evaluar modelos individuales.
- En este aprendizaje se suelen utilizar algoritmos rápidos, como los árboles de decisión, para compensar la demanda computacional.

Ensemble theory

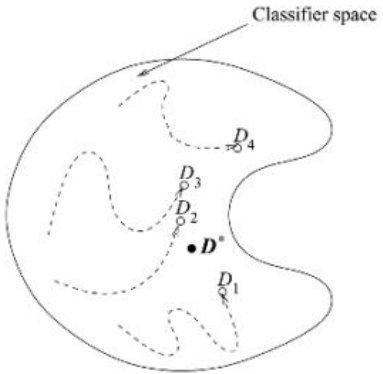
- Un **ensamble** es en sí mismo un algoritmo de aprendizaje supervisado, porque puede entrenarse y luego utilizarse para hacer predicciones.
- Por lo tanto, el **ensamble** entrenado representa una única hipótesis.
- Esta hipótesis no está necesariamente **contenida** en el espacio de hipótesis de los modelos a partir de los cuales se construye.
- Así, se puede demostrar que los **ensambles** tienen más flexibilidad en las funciones que pueden representar.
- Sin embargo, pueden **sobreajustar** los datos **más** que un modelo único.

The statistical reason



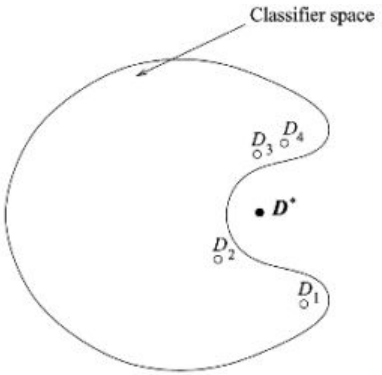
- D* es el mejor clasificador para el problema, la curva exterior muestra el espacio de todos los clasificadores.
- El área sombreada es el espacio de los clasificadores con buenos resultados en los datos de entrenamiento.

The computational reason



- D* es el mejor clasificador para el problema, la curva exterior muestra el espacio de todos los clasificadores.
- Las líneas sombreadas son las trayectorias hipotéticas de los clasificadores durante el entrenamiento.

The representational reason



- D* es el mejor clasificador para el problema, la curva exterior muestra el espacio de todos los clasificadores.
- La forma cerrada muestra el espacio elegido por los clasificadores.

El jurado de Condorcet

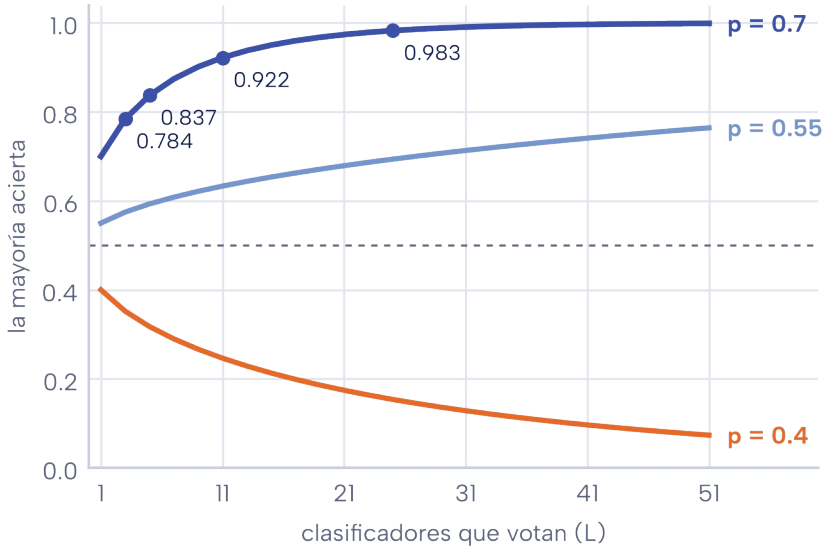
- *L* clasificadores independientes;
  cada uno acierta con probabilidad *p*.
  La mayoría acierta si aciertan más de *L*/2:

$$P_{\text{may}} = \sum_{k > L/2}^{L} \binom{L}{k} p^k (1-p)^{L-k}$$

- Con *L* = 3 y *p* = 0.7:

$$P_{\text{may}} = 3(0.7)^2(0.3) + (0.7)^3 = 0.441 + 0.343 = 0.784$$

- Con *p* = 0.7: 5 → 0.837; 11 → 0.922; **25 → 0.983**.
  Con *p* = 0.4, la mayoría empeora: **25 → 0.154**.



Dos condiciones: cada uno, mejor que el azar, y errores independientes. La segunda casi nunca se cumple.

Si se equivocan juntos, votar no ayuda

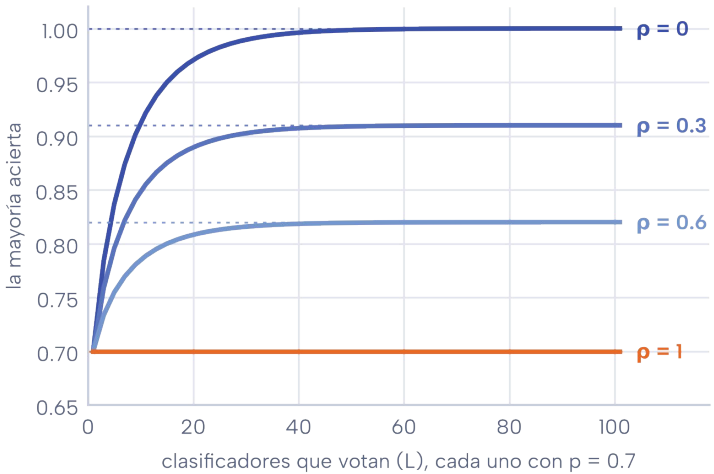
- Con probabilidad ρ, los *L* copian una misma decisión, que acierta con *p*; con 1 − ρ, votan independientes:

$$P = \rho\, p + (1 - \rho)\, P_{\text{may}}(L)$$

- Con $p = 0.7$ y $\rho = 0.3$, ni con infinitos clasificadores se pasa de:

$$0.3(0.7) + 0.7(1) = 0.91$$

- En el vino, los cinco de la unidad 4 fallan juntos en 58 de 320 vinos de prueba (18 %); independientes, serían 3. Correlación media de sus errores: 0.54.



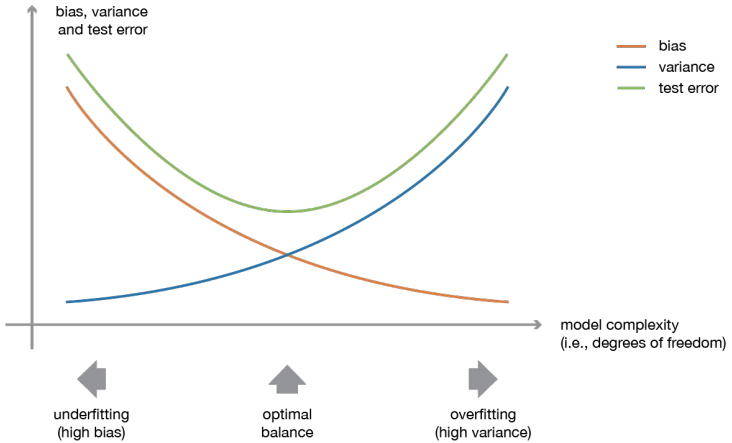
Por eso los ensambles buscan diversidad: datos distintos (bagging), variables distintas (random forest) o modelos distintos (votación y stacking).

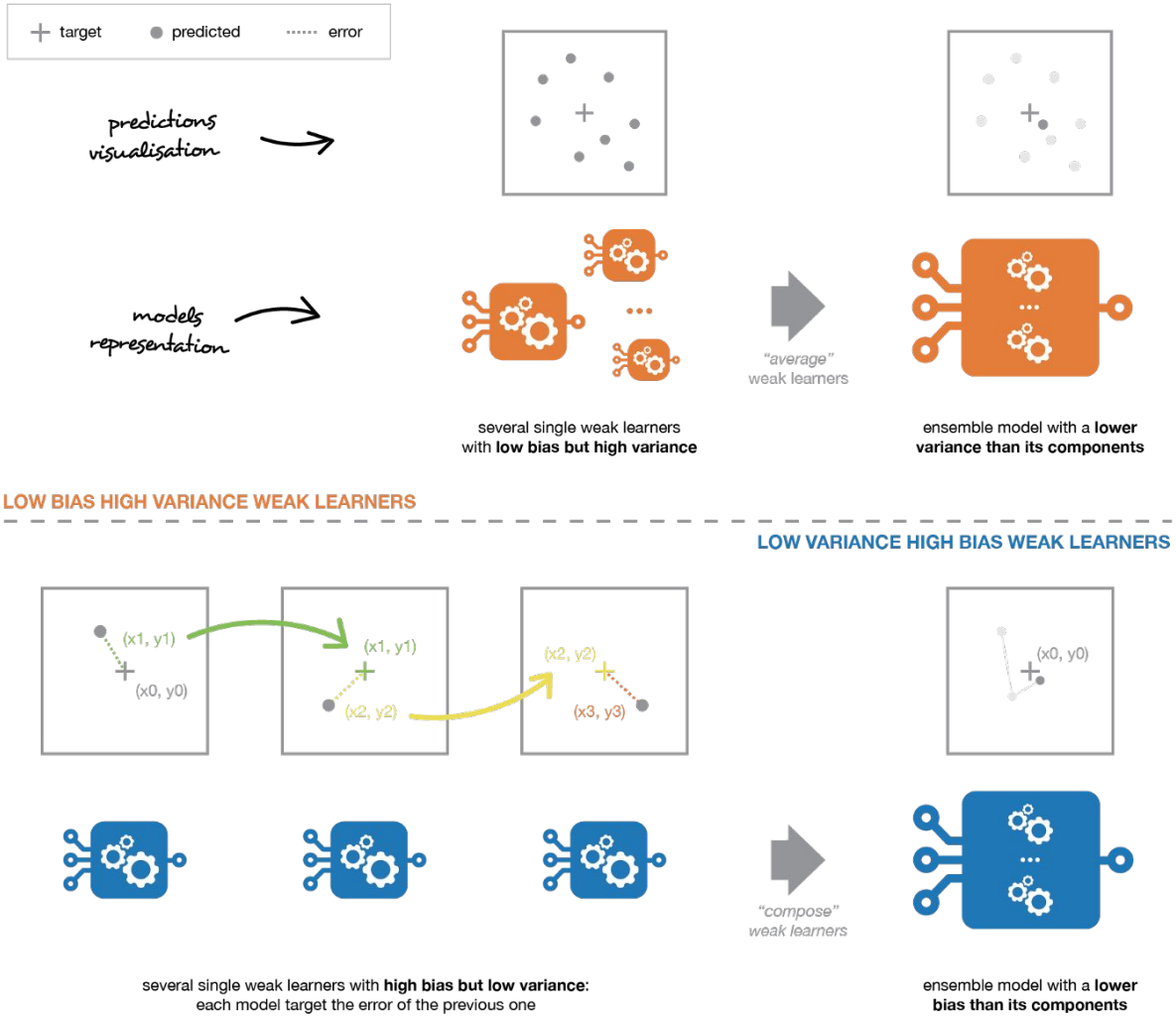
Combine weak learners

- Un método basado en aprendizaje de ensambles necesita **modelos base** para ser agregado.
  - Por lo general, se utiliza un **único algoritmo** de aprendizaje de base, de modo que los aprendices débiles homogéneos se entrenan de distintas formas, por lo que el modelo de **ensamble** es "homogéneo".
  - Sin embargo, también existen algunos métodos que utilizan **diferentes** tipos de **algoritmos** de aprendizaje de base, construyendo un modelo de **ensamble** "heterogéneo".

Combine weak learners

- Para los modelos base con **bajo sesgo** pero **alta varianza**, se utiliza un método de agregación que tiende a **reducir la varianza**.
- Mientras que para los modelos base con **baja varianza** pero **alto sesgo**, se utiliza un método de agregación para **reducir el sesgo**.




target
predicted
error
predictions visualisation
models representation
"average" weak learners
several single weak learners with low bias but high variance
ensemble model with a lower variance than its components
LOW BIAS HIGH VARIANCE WEAK LEARNERS
LOW VARIANCE HIGH BIAS WEAK LEARNERS
(x1, y1)
(x0, y0)
(x1, y1)
(x2, y2)
(x2, y2)
(x3, y3)
(x0, y0)
"compose" weak learners
several single weak learners with high bias but low variance: each model target the error of the previous one
ensemble model with a lower bias than its components

Tipos comunes de ensambles

Método	Modelos base	Se entrenan	Se combinan con	Reduce	Cuándo
Combinación bayesiana	hipótesis de H	—	un promedio pesado por $P(h \mid D)$	—	hoy y sesión 14
Votación	distintos	por separado, con los mismos datos	un voto, duro o suave	varianza	hoy
Stacking	distintos	en paralelo; el metamodelo, con predicciones fuera del pliegue	un metamodelo	varianza y sesgo	hoy
Bagging	iguales	en paralelo, en muestras bootstrap	un voto o un promedio	varianza	hoy
Boosting	iguales	en secuencia: cada uno corrige a los anteriores	una suma ponderada	sesgo	siguiente sesión

La votación es el stacking más simple: pesos fijos en lugar de un metamodelo que los aprende.

Clasificador óptimo de Bayes

- Sesión 14: la clasificación más probable de *x* no es la de h_{MAP}. Se combinan todas las hipótesis de *H*, cada una pesada por su posterior:

$$v_{OB} = \arg\max_{v_j \in V} \sum_{h_i \in H} P(v_j \mid h_i)\, P(h_i \mid D)$$

- Con $P(h_1 \mid D) = 0.4$, $P(h_2 \mid D) = P(h_3 \mid D) = 0.3$, y $h_1(x) = +$, $h_2(x) = h_3(x) = -$:

$$\sum_i P(+ \mid h_i)\, P(h_i \mid D) = 1(0.4) + 0(0.3) + 0(0.3) = 0.4$$

$$\sum_i P(- \mid h_i)\, P(h_i \mid D) = 0(0.4) + 1(0.3) + 1(0.3) = 0.6$$

- $h_{MAP} = h_1$ dice **+**; el ensamble dice **−**. Es un voto ponderado por $P(h \mid D)$, y puede representar una hipótesis que no está en *H*.

Promedio bayesiano de modelos (BMA)

- En la práctica, en lugar de sumar sobre todo *H*, se toma una muestra *M* de hipótesis (Monte Carlo) y se pesan con el teorema de Bayes.
  La posterior es proporcional, no igual, a la verosimilitud por la previa:

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)} \;\propto\; P(D \mid h)\, P(h)$$

$$v_{BMA} = \arg\max_{v_j \in V} \sum_{h \in M} P(v_j \mid h)\, P(D \mid h)\, P(h)$$

- El algoritmo de Gibbs toma una sola hipótesis al azar, según *P*(h | D). Bajo ciertas condiciones, su error esperado es a lo más el doble del óptimo de Bayes.
- Con muchos datos, *P*(D | h) se concentra en una hipótesis y el promedio se queda con un solo modelo.

Combinación bayesiana de modelos (BMC)

- El promedio bayesiano supone que una de las hipótesis es la correcta y, con más datos, busca cuál: es elegir, no combinar.
- La combinación bayesiana muestrea ponderaciones del ensamble, de una distribución de Dirichlet, en lugar de hipótesis sueltas, y pesa cada combinación por su posterior. Es más cara de calcular.
- Exactitud media en sus experimentos:

	Árbol J48	Bagging	Boosting	BMA	BMC
exactitud media (%)	82.37	82.79	83.62	**81.64**	**84.02**

Votación dura y suave

- **Dura**: cada modelo vota por su clase y gana la más votada.

$$\hat{y} = \arg\max_{k} \left| \{\, l : \hat{y}_l = k \,\} \right|$$

- **Suave**: se promedian las probabilidades y gana la mayor.

$$\hat{y} = \arg\max_{k} \frac{1}{L} \sum_{l=1}^{L} P_l(k \mid x)$$

- En los 320 vinos de prueba difieren en 17. F1 macro: dura 0.595, suave 0.572; en validación, 0.588 y 0.591.

	Vino 249: es **baja**			**Vino 218:** es **alta**		
	baja	**media**	**voto**	**alta**	**media**	**voto**
logística	0.48	0.50	media	0.47	0.44	alta
árbol elegido	0.38	0.54	media	0.04	0.92	media
Naive Bayes	0.93	0.07	baja	0.58	0.39	alta
kNN	0.34	0.66	media	0.42	0.36	alta
SVM	0.32	0.60	media	0.54	0.16	alta
promedio	0.49	0.47		0.41	0.45	
dura			**media 4–1**			**alta 4–1**
suave			**baja**			**media**

Stacking

- *Stacking* difiere del *bagging* y del *boosting* principalmente en dos puntos.
  - En primer lugar, el apilamiento suele tener en cuenta aprendices débiles **heterogéneos** (se combinan distintos algoritmos de aprendizaje), mientras que *bagging* y *boosting* tienen en cuenta aprendices débiles **homogéneos**.
  - En segundo lugar, el stacking aprende a combinar los modelos base utilizando un **metamodelo**, mientras que el bagging y el boosting combinan los aprendices débiles siguiendo algoritmos deterministas.

Stacking

Algorithm 1 - Stacking

Input : $D = \{(x_i, y_i) | x_i \in \chi,\ y_i \in Y\}$

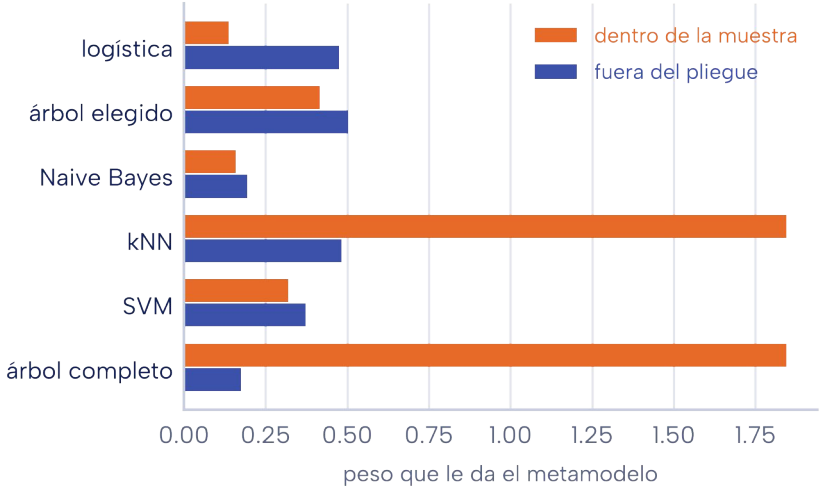
Output : An ensemble classifier H

1. **Step 1 :** Learn first-level classifiers
2. For $t \leftarrow 1$ to T do
3. Learn a base classifier h_t based on D
4. **Step 2 :** Construct new data set from D
5. For $i \leftarrow 1$ *to* m do
6. Construct a new data set that contains $\{x_i^{new}, y_i\}$, where $x_i^{new} = \{h_j(x_i)\ for\ j = 1\ to\ T\}$
7. **Step 3 :** Learn a second-level classifier
8. Learn a new classifier h^{new} based on the newly constructed data set
9. **Return** $H(x) = h^{new}(h_1(x), h_2(x), \ldots, h_T(x))$

Stacking sin fugas

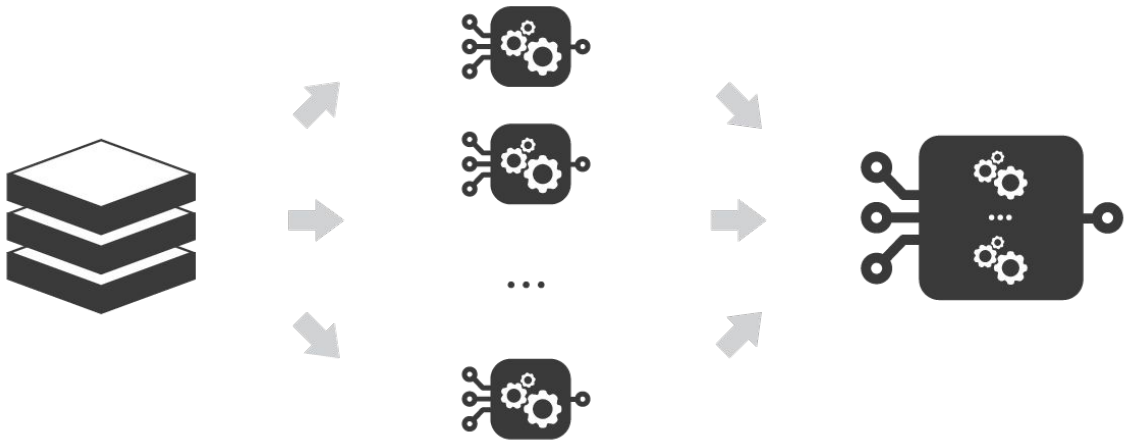
- En el paso 2 del algoritmo, el metamodelo aprende de las predicciones de modelos que ya vieron esos mismos datos: predicciones demasiado buenas.
- Por eso cada dato recibe la predicción de modelos entrenados sin él: predicciones fuera del pliegue, agrupadas en el vino. `StackingClassifier` lo hace con su parámetro `cv`.

F1 macro en prueba	dentro de la muestra	fuera del pliegue
los cinco de la unidad 4	0.597	0.573
los cinco + árbol completo	**0.488**	**0.567**



Dentro de la muestra, el metamodelo premia a quien memoriza: el kNN y el árbol completo. El kNN generaliza bien y no se nota; el árbol completo hunde al ensamble. Fuera del pliegue, recibe el peso más bajo.

Stacking



initial dataset

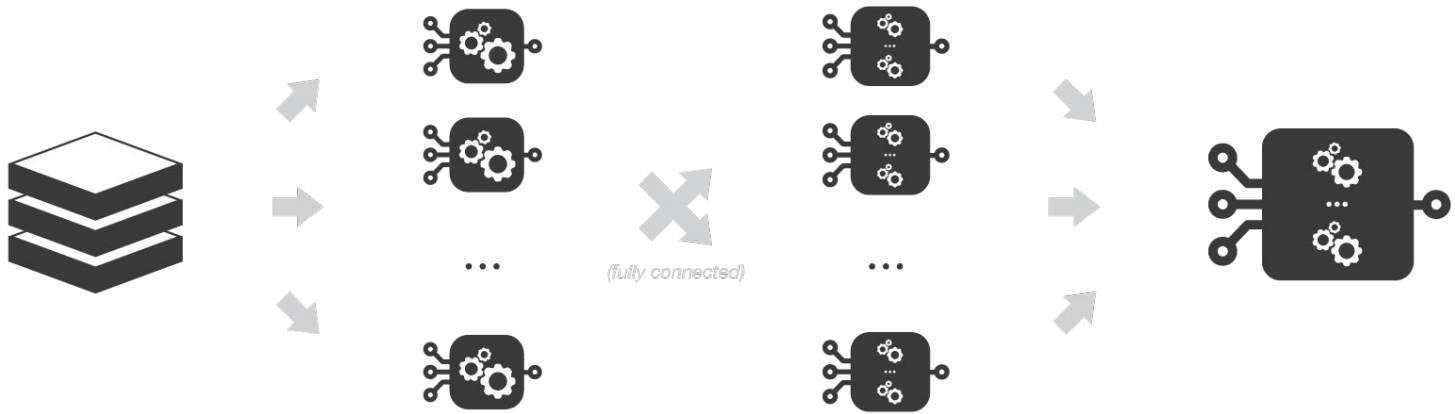
L weak learners
(that can be non-homogeneous)

meta-model
(**trained** to output predictions based
on weak learners predictions)

Multi-levels Stacking

- Consiste en hacer apilamientos con múltiples capas.
- Considere un *stacking* de 3 niveles:
  - En el primer nivel (capa), ajustar los L aprendices débiles.
  - A continuación, en lugar de ajustar un único metamodelo sobre las predicciones de los modelos débiles, ajustar M metamodelos de este tipo.
  - Por último, ajustar un último metamodelo que toma como entradas las predicciones devueltas por los M metamodelos.
- La adición de niveles puede ser costosa en datos (si no se utiliza la técnica k-folds y, por tanto, se necesitan más datos) o en tiempo (si se utiliza la técnica k-folds y, por tanto, es necesario ajustar muchos modelos).

Multi-levels Stacking



initial dataset

L weak learners
(that can be non-homogeneous)

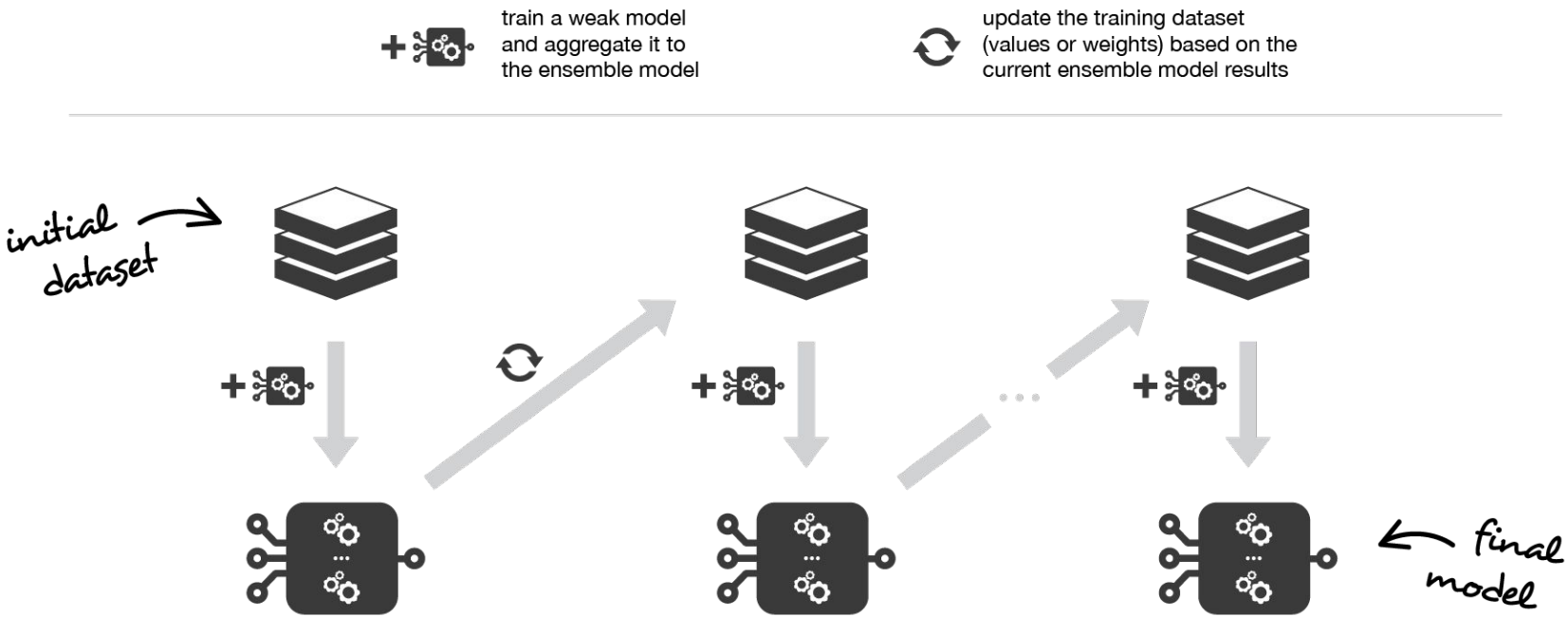
M meta-models
(**trained** to output predictions based on previous layer predictions)

final meta-model
(**trained** to output predictions based on previous layer predictions)

Boosting

- El *boosting* construye un conjunto de **forma incremental**:
  cada nuevo modelo se entrena para enfatizar los casos que los modelos anteriores clasificaron mal.
- En muchos problemas, *boosting* ofrece mayor precisión que *bagging*, pero es más sensible al ruido: los ejemplos mal etiquetados acumulan peso ronda tras ronda.
- La primera implementación práctica fue **AdaBoost**; hoy dominan las de *gradient boosting*: XGBoost, LightGBM y CatBoost.
- Los modelos se combinan con un voto ponderado por su desempeño (sesión 18).

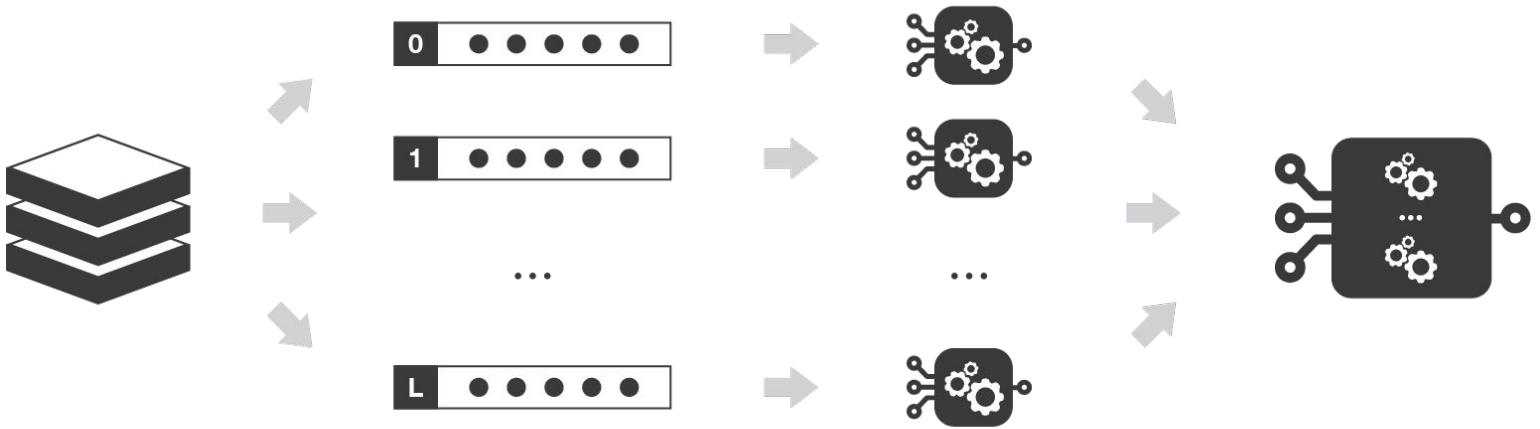
Boosting


train a weak model
and aggregate it to
the ensemble model
update the training dataset
(values or weights) based on the
current ensemble model results
initial
dataset
final
model

Bagging

- *Bootstrap aggregating* (*bagging*) implica que cada **modelo débil** del conjunto vote con el mismo peso.
- Para promover la varianza de los modelos, se entrena a cada modelo del conjunto utilizando un **subconjunto** del conjunto **de entrenamiento** extraído aleatoriamente.
- Las muestras se generan de forma que sean diferentes entre sí, aunque se permite el reemplazo.
- Estas muestras se entregan a múltiples modelos y los resultados de cada uno se combinan en forma de votación.

Bagging



initial dataset

L bootstrap samples

weak learners fitted on each bootstrap sample

ensemble model (kind of average of the weak learners)

5.2 Bagging

Centrarse en los bootstraps

- En los **métodos paralelos**, los diferentes modelos base se ajustan de forma independiente entre sí y es posible entrenarlos simultáneamente.
- El enfoque más famoso es el *bagging (bootstrap aggregating)*, o también conocido como "agregación de arranque".
- El *bagging* produce un modelo ensamble que es más robusto que los modelos individuales que lo componen.

Bootstrapping

- Esta técnica estadística consiste en generar muestras de tamaño *B* (denominadas muestras *bootstrap*) a partir de un conjunto de datos inicial de tamaño *N* extrayendo aleatoriamente B observaciones con reemplazo.



En bagging, B=N: cada muestra tiene el tamaño del conjunto original.

Bootstrapping

- Bajo ciertas suposiciones, estas **muestras** tienen buenas propiedades estadísticas: pueden considerarse muestras representativas e independientes de la distribución real de los datos.
- Las hipótesis a verificar para que esta aproximación sea válida son dos:
  - **Representatividad**: *N* debe ser lo suficientemente grande como para capturar la mayor parte de la complejidad de la distribución subyacente.
  - **Independencia**: *N* debe ser lo suficientemente grande en comparación con *B* para que las muestras no estén demasiado correlacionadas. En bagging, B = N y las muestras sí se traslapan: dos de ellas comparten en promedio 40% de los datos (diapositiva 38).

Una muestra bootstrap de los 14 días



- Cada muestra saca 14 días con reemplazo ($B = N$): unos se repiten y otros no salen. La muestra 1 tiene 8 días distintos; quedaron fuera 1, 4, 5, 6, 9 y 12.
- Un día no sale con probabilidad 0.354: en promedio, 4.96 días quedan fuera de la bolsa (*out-of-bag*).

$$\left(1 - \tfrac{1}{14}\right)^{14} = \left(\tfrac{13}{14}\right)^{14} = 0.354$$

- Con N grande, cada muestra deja fuera 36.8 % de los datos y usa el 63.2 % restante.

$$\left(1 - \tfrac{1}{N}\right)^{N} \longrightarrow e^{-1} = 0.368$$

Bootstrap samples

- Las muestras *bootstrap* se utilizan a menudo para evaluar la varianza o los intervalos de confianza de los estimadores estadísticos.
- Por definición, un **estimador estadístico** es una función de algunas observaciones, i.e., una variable aleatoria con varianza procedente de estas observaciones.
- Para estimar la varianza de un estimador de este tipo, se evalúan varias muestras independientes extraídas de la distribución de interés.

Bootstrap samples

- En la mayoría de los casos, considerar **muestras verdaderamente independientes** requeriría demasiados datos en comparación con la cantidad realmente disponible.
- El *bootstrapping* se utiliza para generar varias muestras *bootstrap* que pueden considerarse "casi-representativas" y "casi-independientes".
- Estas muestras *bootstrap* **aproximan** la varianza del estimador, evaluando su valor para cada una de ellas.



initial dataset

L bootstrap samples

estimator of interest evaluated for each bootstrap sample

variance and confidence intervals computed based on the L realisations of the estimator

Bagging

- Cuando se entrena un modelo, ya sea para problemas de clasificación o de regresión, una función (modelo) devuelve un resultado que se define con respecto al conjunto de datos de entrenamiento.
  - Debido a la **varianza teórica** del conjunto de datos de entrenamiento, el modelo ajustado también está sujeto a la **variabilidad**: si se observa otro conjunto de datos, se construye un modelo diferente.
- La idea del *bagging* es sencilla: ajustar varios modelos independientes y "promediar" sus predicciones para obtener un modelo con una varianza menor.
  - Sin embargo, en la práctica, ajustar modelos totalmente independientes requiere demasiados datos.

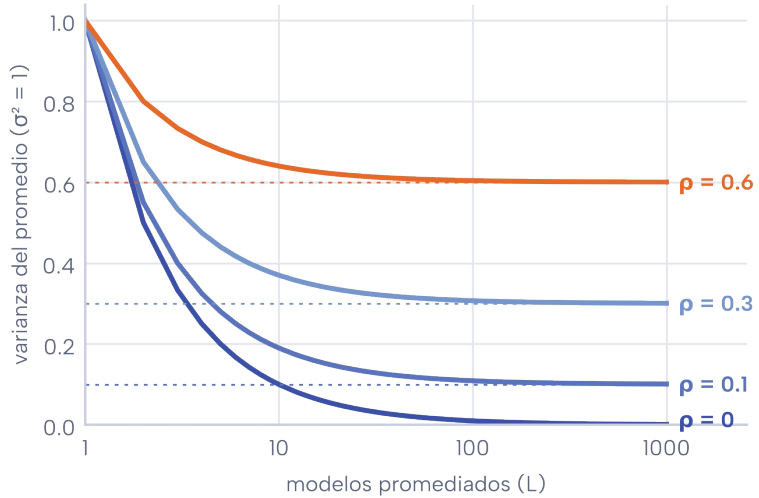
Bagging

- Por tanto, debemos confiar en las buenas "propiedades aproximadas" de las muestras *bootstrap* (representatividad e independencia) para ajustar modelos que sean casi independientes.
  - Crear múltiples muestras *bootstrap* para que cada una de ellas actúe como otro conjunto de datos (casi) independiente extraído de la distribución real.
- Ajustar un modelo base para cada una de estas muestras y, finalmente, agregarlos para obtener un modelo de ensamble con menos varianza que sus componentes.
  - Promediar los resultados de los modelos base no cambia la respuesta esperada, pero reduce su varianza; cuánto, depende de qué tan correlacionados estén (siguiente diapositiva).

¿Cuánto baja la varianza al promediar?

$$\mathrm{Var}\left(\frac{1}{L}\sum_{l=1}^{L} f_l\right) = \frac{1}{L^2}\left[L\sigma^2 + L(L-1)\,\rho\sigma^2\right] = \rho\,\sigma^2 + \frac{1-\rho}{L}\,\sigma^2$$

- L modelos, cada uno con varianza σ^2, y correlación ρ entre pares.
- Si fueran independientes ($\rho = 0$), bajaría como $1/L$. Con muestras bootstrap de los mismos datos no lo son: dos de ellas comparten en promedio $(1 - e^{-1})^2 \approx 40\ \%$ de los datos.
- Con $\rho = 0.3$: $L = 10 \rightarrow 0.37$; $L = 100 \rightarrow 0.307$; nunca baja de 0.3. Más modelos solo achican el segundo término; el primero baja si baja ρ: es la idea de **random forest**.



Bagging

- Sean *L* muestras bootstrap (aproximaciones de *L* conjuntos de datos independientes) de tamaño *B* = *N*:

$$\left\{z_1^1, z_2^1, \ldots, z_N^1\right\},\ \left\{z_1^2, z_2^2, \ldots, z_N^2\right\},\ \ldots,\ \left\{z_1^L, z_2^L, \ldots, z_N^L\right\}$$

- donde z^l_b es la *b*–ésima observación de la *l*–ésima muestra. Se ajusta un modelo base en cada una, casi independientes entre sí:

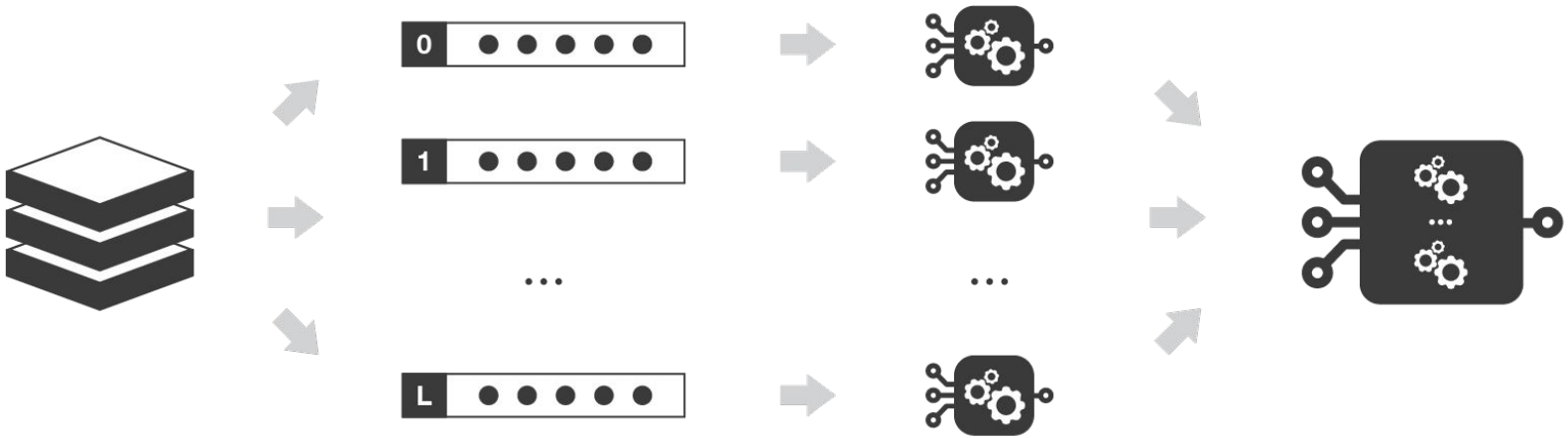
$$w_1(\cdot),\ w_2(\cdot),\ \ldots,\ w_L(\cdot)$$

- y se agregan en algún tipo de promedio, que tiene menor varianza:

regresión $s_L(x) = \frac{1}{L}\sum_{l=1}^{L} w_l(x)$ *clasificación* $s_L(x) = \arg\max_k \left|\left\{\, l : w_l(x) = k \,\right\}\right|$

Bagging

- Hay varias formas de agregar múltiples modelos ajustados en paralelo.
  - Para **regresión**, los resultados de los modelos individuales pueden promediarse, literalmente, para obtener el resultado del modelo de conjunto.
  - Para **clasificación**, la salida de cada modelo es un voto y la clase que recibe la mayoría de los votos es devuelta por el modelo de conjunto: *hard-voting*.
  - Si se tienen en cuenta las probabilidades de cada clase obtenidas por todos los modelos, se calcula la media de estas probabilidades y se mantiene la clase con la probabilidad media más alta: *soft-voting*.



initial dataset

L bootstrap samples

weak learners fitted on each bootstrap sample

ensemble model (kind of average of the weak learners)

Bagging a mano: tres árboles, 14 días

Muestra 1

días: 2 2 2 3 7 7 8 10 10 11 11 11 13 14

fuera: 1 4 5 6 9 12

```
Humidity = high
      Outlook = overcast: Sí [1+, 0−]
      Outlook = rain: No [0+, 1−]
      Outlook = sunny: No [0+, 4−]
Humidity = normal: Sí [8+, 0−]
```

en prueba: **10/14**

Muestra 2

días: 2 3 6 6 7 8 8 10 11 11 12 12 12 13

fuera: 1 4 5 9 14

```
Outlook = overcast: Sí [6+, 0−]
Outlook = rain
      Temperature = cool: No [0+, 2−]
      Temperature = hot: No [0+, 0−]
      Temperature = mild: Sí [1+, 0−]
Outlook = sunny
      Humidity = high: No [0+, 3−]
      Humidity = normal: Sí [2+, 0−]
```

en prueba: **13/14**

Muestra 3

días: 1 2 3 4 4 7 7 8 8 9 11 12 13 13

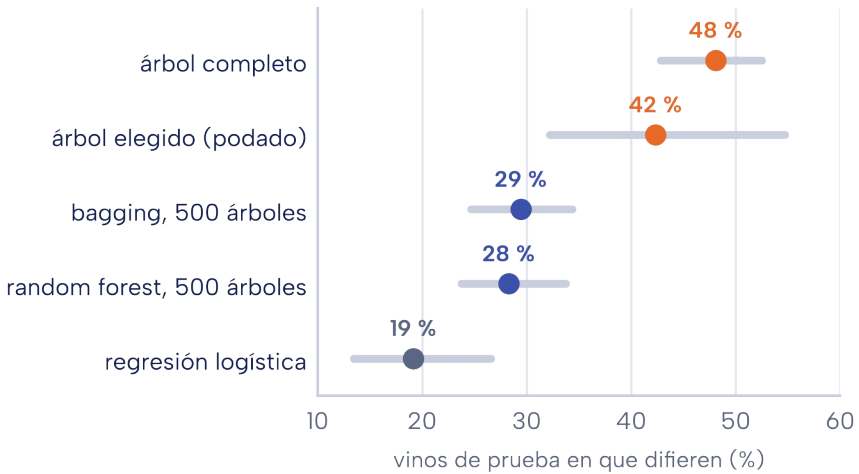
fuera: 5 6 10 14

```
Outlook = overcast: Sí [6+, 0−]
Outlook = rain: Sí [2+, 0−]
Outlook = sunny
      Humidity = high: No [0+, 4−]
      Humidity = normal: Sí [2+, 0−]
```

en prueba: **11/14**

día	**15**	**16**	**17**	**18**	**19**	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**
árbol 1	No	Sí	**No**	**Sí**	No	No	Sí	**No**	Sí	Sí	Sí	Sí	**Sí**	Sí
árbol 2	No	Sí	Sí	**Sí**	No	No	Sí	Sí	Sí	Sí	Sí	Sí	No	Sí
árbol 3	No	Sí	Sí	**Sí**	No	**Sí**	Sí	Sí	Sí	Sí	Sí	Sí	**Sí**	Sí
voto	No	Sí	Sí	**Sí**	No	No	Sí	Sí	Sí	Sí	Sí	Sí	**Sí**	Sí
real	*No*	*Sí*	*Sí*	*No*	*No*	*No*	*Sí*	*Sí*	*Sí*	*Sí*	*Sí*	*Sí*	*No*	*Sí*

Bagging en el vino: el árbol se estabiliza

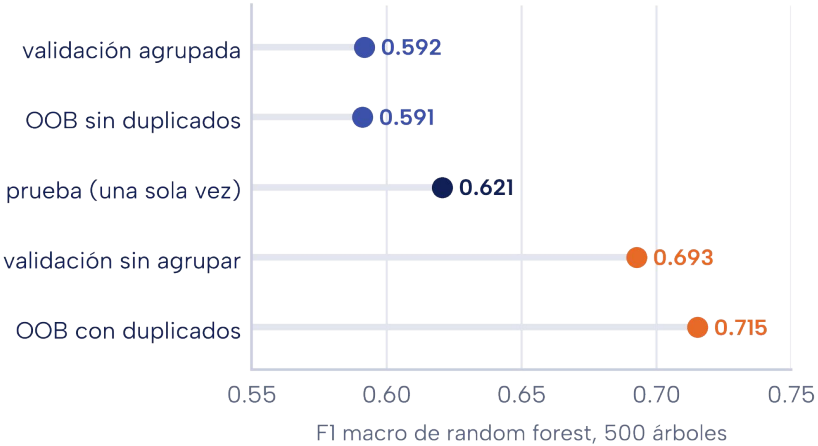


- El experimento del notebook 13: dos modelos entrenados con mitades distintas del entrenamiento (las gemelas juntas), 20 veces. ¿En cuántos vinos de prueba difieren?
- Dos árboles completos difieren en 48 %; dos bosques de 500 árboles, en 29 % (bagging) y 28 % (random forest). La logística sigue más estable: 19 %.
- Y aciertan más: con media muestra, el F1 macro sube de 0.494 (árbol completo) a 0.577 y 0.584.

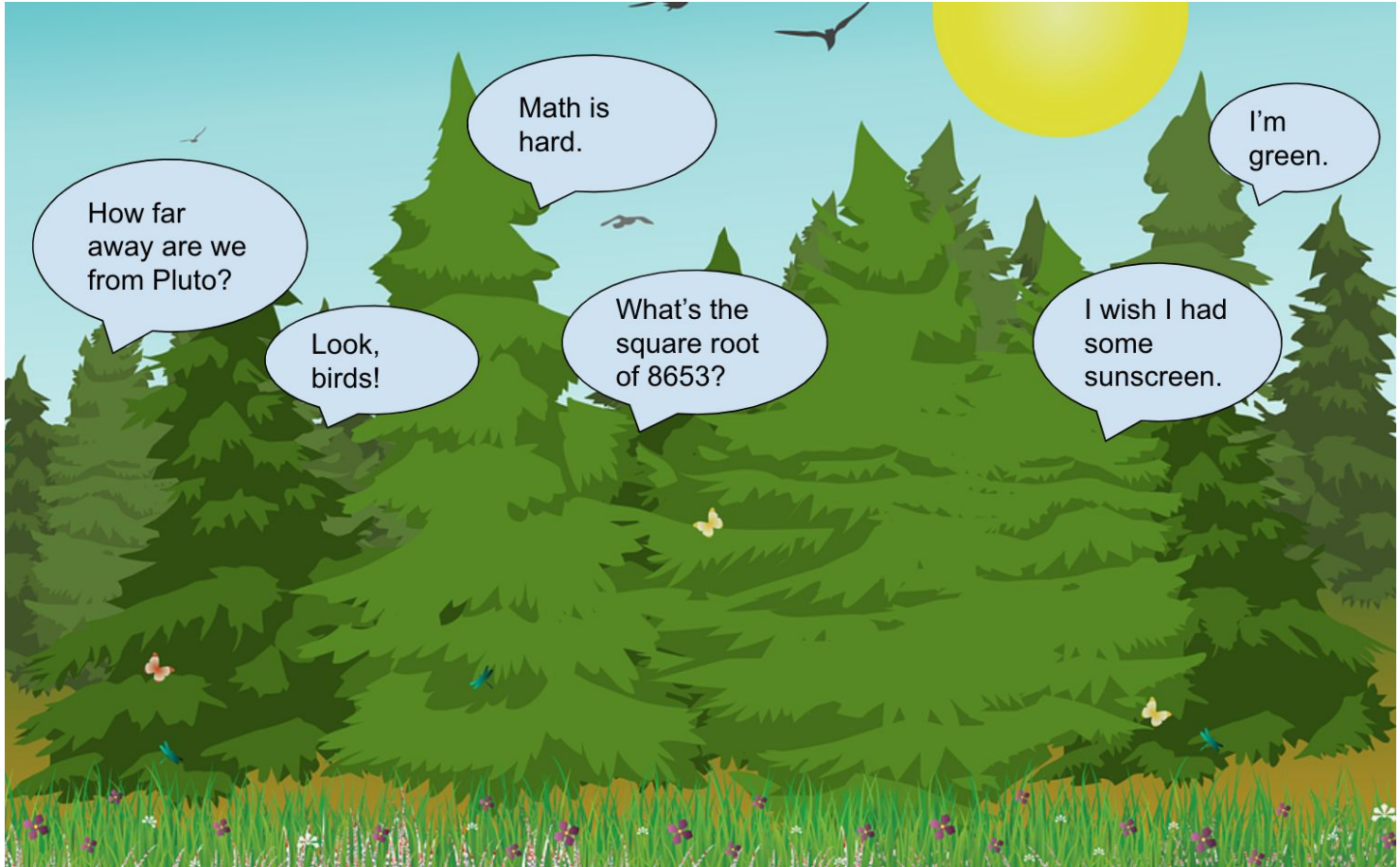
Cada bosque promedia árboles de muestras de su misma mitad: hereda lo que esa mitad tiene de particular. Por eso el desacuerdo no baja a cero.

El error fuera de la bolsa (OOB)

- Cada vino queda fuera de cerca de 37 % de las muestras: esos árboles no lo vieron. Votando solo con ellos se tiene una validación gratis, el error OOB (`oob_score=True`).
- En el vino, con 500 árboles: OOB de 0.715 de F1 macro. ¿Por qué tan optimista?
- 366 de los 1279 vinos de entrenamiento tienen una gemela (sesión 12). Si la gemela está en la muestra, el árbol la memorizó: el OOB acierta en 100 % de esos vinos y en 64 % del resto.
- Sin duplicados (1088 vinos), el OOB da 0.591: igual que la validación agrupada.



Just a random forest...



Random Forest

- Como ya se mencionó, los **árboles de decisión** son modelos base muy populares para los métodos de ensamble.
- Los 'aprendices fuertes' compuestos por varios árboles se denominan bosques (***forests***).
- Los árboles que componen un bosque pueden ser superficiales o *shallow* (poca profundidad), o profundos (mucha profundidad).
    - Los árboles poco profundos tienen menos varianza pero mayor sesgo, por lo que son la mejor opción para los **métodos secuenciales**.
    - Los árboles profundos tienen un sesgo bajo, pero una varianza alta, por lo que son una opción adecuada para *bagging*.

Random Forest

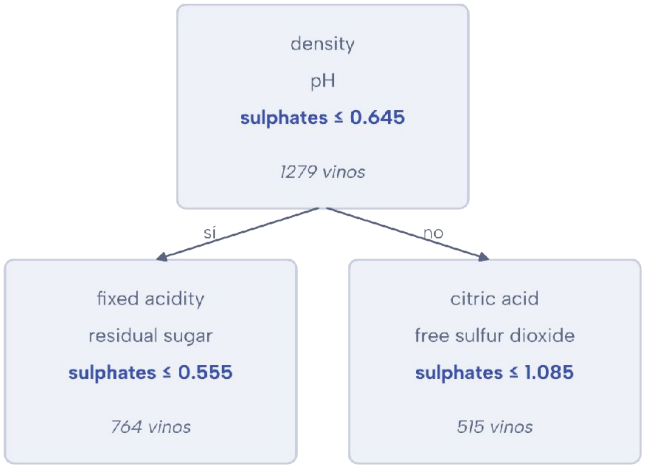
- El enfoque de *random forest* combina árboles profundos, ajustados en muestras *bootstrap*, para producir un resultado con menor varianza.
- Y usa otro truco para que los árboles estén menos correlacionados:
  - **en cada nodo**, al buscar el mejor corte, sortea `max_features` de las D variables y solo considera esas. El nodo siguiente sortea otras.
  - Por omisión, scikit-learn usa $\sqrt{D}$ en clasificación: 3 de las 11 variables del vino.
- Sortear las variables una sola vez por árbol es otro método: los subespacios aleatorios.

Random Forest

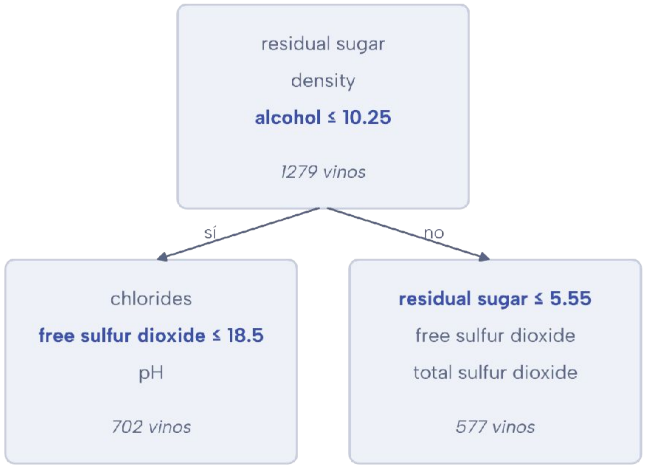
- Sortear las variables en cada nodo hace que los árboles no hagan siempre las mismas preguntas y baja la correlación entre sus resultados.
- Cada árbol puede usar todas las variables, cada una en nodos distintos.
- **Datos faltantes**: desde la versión 1.4, `RandomForestClassifier` acepta `NaN`. En cada corte prueba mandar los faltantes a cada lado y se queda con lo mejor, como el árbol de la sesión 13.
- *Random forest* combina dos fuentes de azar:
  - las muestras *bootstrap* (*bagging*) y
  - la selección aleatoria de variables en cada nodo.

Random Forest

Árbol 1: muestra bootstrap con 802 vinos distintos



Árbol 2: muestra bootstrap con 812 vinos distintos



- En cada nodo se sortean 3 de las 11 variables (gris) y gana el mejor corte de Gini entre ellas (azul).
- En el árbol 1, alcohol, el mejor corte para la raíz, no salió en ningún sorteo: ganó sulphates, el segundo. Así los árboles hacen preguntas distintas.

Random Forest algorithm

Algorithm 1: Pseudo code for the random forest algorithm

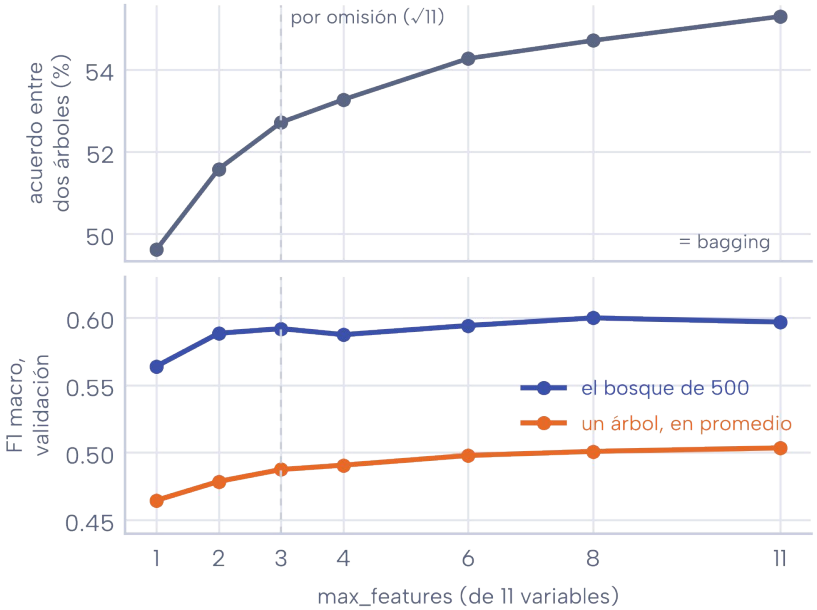
To generate c classifiers:
for i = 1 to c **do**
  Randomly sample the training data D with replacement to produce D_i
  Create a root node, N_i containing D_i
  Call BuildTree(N_i)
end for

BuildTree(N):
if N contains instances of only one class **then**
  return
else
  Randomly select x% of the possible splitting features in N
  Select the feature F with the highest information gain to split on
  Create f child nodes of N, N_1,..., N_f, where F has f possible values (F_1, … , F_f)
  for i = 1 to f **do**
    Set the contents of N_i to D_i, where D_i is all instances in N that match
    F_i
    Call BuildTree(N_i)
  end for
end if

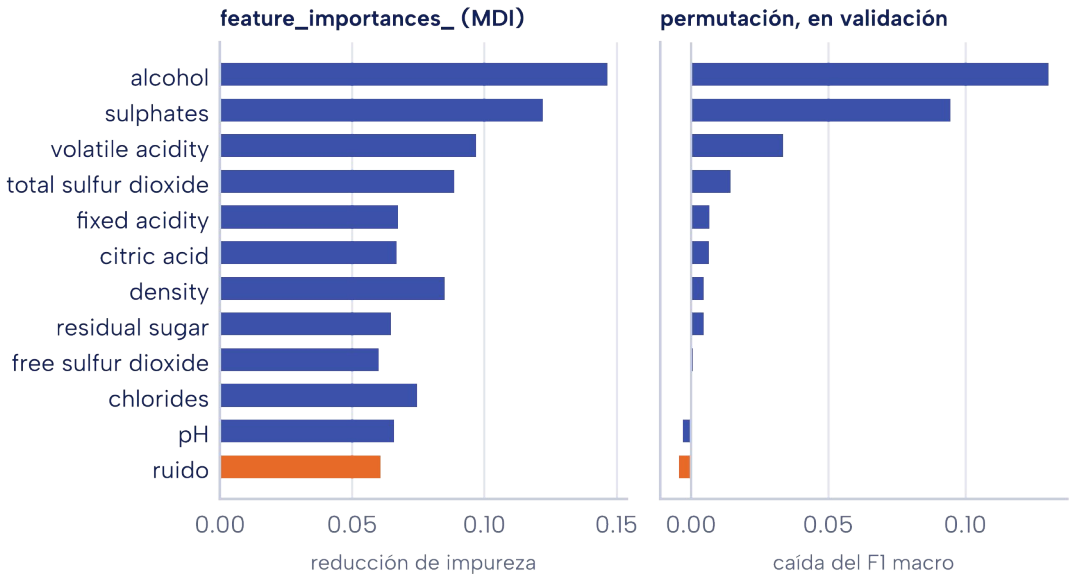
¿Cuántas variables sortear?

- Con `max_features` = 11 es bagging: dos árboles del bosque coinciden en 55 % de los vinos de validación; con 1, en 50 %.
- Pero con menos variables por nodo, cada árbol acierta menos: su F1 macro baja de 0.503 a 0.465.
- El bosque compensa: de 2 a 11 queda entre 0.587 y 0.600, con desviación entre pliegues de 0.04 a 0.06. En el vino, `max_features` casi no importa; solo con 1 baja (0.564).

El error del bosque depende de la fuerza de cada árbol y de la correlación entre ellos.



¿Qué variables importan?



- `feature_importances_` (MDI) suma cuánto reduce la impureza cada variable en todos los nodos. Se calcula con los datos de entrenamiento.
- Una columna de ruido puro queda en el lugar 11 de 12 con MDI (0.061), junto a variables reales: los árboles profundos la usan para memorizar.
- La permutación mide cuánto baja el F1 en validación al barajar la variable: el ruido queda en 0.

En validación, solo alcohol, sulphates y volatile acidity pesan claramente.

Variantes de Random Forest

Variante	Qué se sortea	F1 validación	F1 prueba
Bagging	las filas (bootstrap)	0.592 ± .039	0.618
Random forest	las filas y, en cada nodo, las variables	0.592 ± .043	0.621
Extra-Trees	en cada nodo, las variables y el umbral; sin bootstrap	0.573 ± .053	0.642
Subespacios aleatorios	la mitad de las variables, una vez por árbol	0.555 ± .044	0.611
Parches aleatorios	la mitad de las filas y de las variables, por árbol	0.557 ± .056	0.538

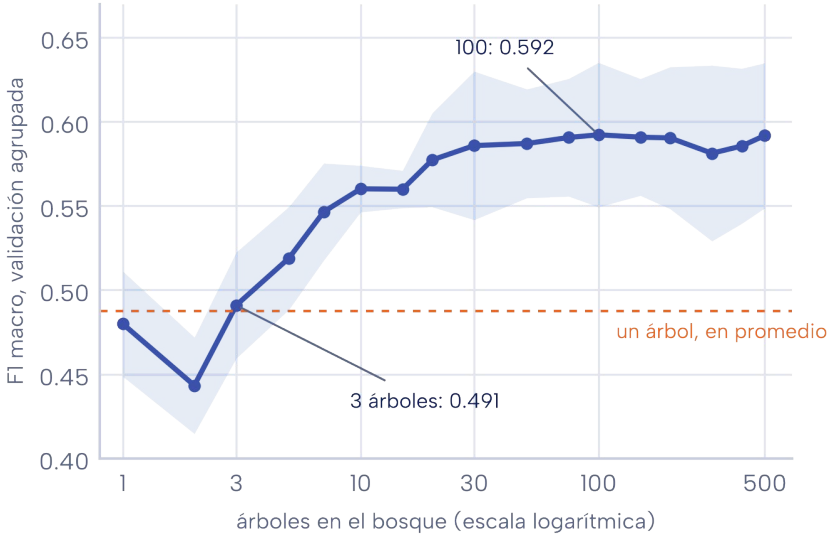
- Extra-Trees sortea también el umbral: árboles aún más distintos y más rápidos de entrenar.
- En el vino empatan: las diferencias en validación caben en la desviación entre pliegues.
  El 0.642 de Extra-Trees en prueba se ve una sola vez y no sirve para elegir.

Resumen de Bagging

- *Bagging* entrena el mismo modelo base en L muestras *bootstrap*, en paralelo, y promedia o vota. Baja la varianza, $\rho\sigma^2 + (1 - \rho)\sigma^2/L$, no el sesgo.
- Le convienen modelos base con sesgo bajo y varianza alta: árboles profundos.
- *Random forest* además sortea `max_features` variables en cada nodo para bajar ρ.
- El error OOB es una validación gratis, salvo si hay duplicados.
- Para la importancia de las variables, mejor la permutación en validación que `feature_importances_`.
- En el vino, el árbol completo pasa de 0.484 a 0.621 en prueba y de 48% a 28% de desacuerdo.

Tamaño de los ensambles

- En *random forest*, agregar árboles no sobreajusta: el error converge. Solo cuesta tiempo.
- En el vino, el F1 de validación sube rápido y se estabiliza cerca de 100 árboles (0.592); de 100 a 500 cambia menos de 0.01.
- Con pocos árboles importa la semilla: con 5, el F1 va de 0.505 a 0.561; con 100, de 0.583 a 0.597.
- «Tantos clasificadores como clases» supone clasificadores independientes; aquí serían 3 árboles: 0.491.
- En *boosting* es distinto: demasiados modelos sí sobreajustan (sesión 18).



Los ensambles en el vino

Modelo	F1 validación	F1 prueba
La unidad 4		
Regresión logística (11)	0.562 ± .022	0.573
Árbol elegido (13)	0.584 ± .022	0.551
Naive Bayes (14)	0.579 ± .044	0.596
kNN (15)	0.566 ± .041	0.584
SVM (16)	0.582 ± .027	0.576
Bagging		
Árbol completo, sin bagging	0.551 ± .038	0.484
Bagging, 500 árboles	**0.592 ± .039**	**0.618**
Random forest, 500 árboles	**0.592 ± .043**	**0.621**
Extra-Trees, 500 árboles	**0.573 ± .053**	**0.642**
Combinar a los cinco		
Votación dura	0.588 ± .022	0.595
Votación suave	0.591 ± .013	0.572
Stacking, fuera del pliegue	0.560 ± .019	0.573

- Los ensambles de árboles arreglan al árbol: de 0.484 a 0.62–0.64 en prueba.
- Contra el mejor de la unidad 4, la ganancia en validación (0.592 contra 0.584) cabe en la desviación entre pliegues.
- Combinar a los cinco no ayuda: se equivocan juntos.
- Siguiente sesión, en la misma tabla: AdaBoost, gradient boosting, XGBoost, LightGBM y CatBoost.

¿Y lo «meta» de meta-aprendizaje?

- En esta unidad, *meta* quiere decir aprender de lo que aprendieron otros modelos: el metamodelo del *stacking* aprende de sus predicciones.
- En la literatura, *meta-learning* casi siempre significa **aprender a aprender**: usar la experiencia de muchas tareas para resolver mejor una nueva.
    - Elegir algoritmo e hiperparámetros a partir de tareas parecidas (AutoML).
    - Aprender con pocos ejemplos (*few-shot learning*).
- TabPFN junta las dos ideas: un *transformer* preentrenado con millones de tablas sintéticas que predice una tabla nueva sin entrenarse con ella.

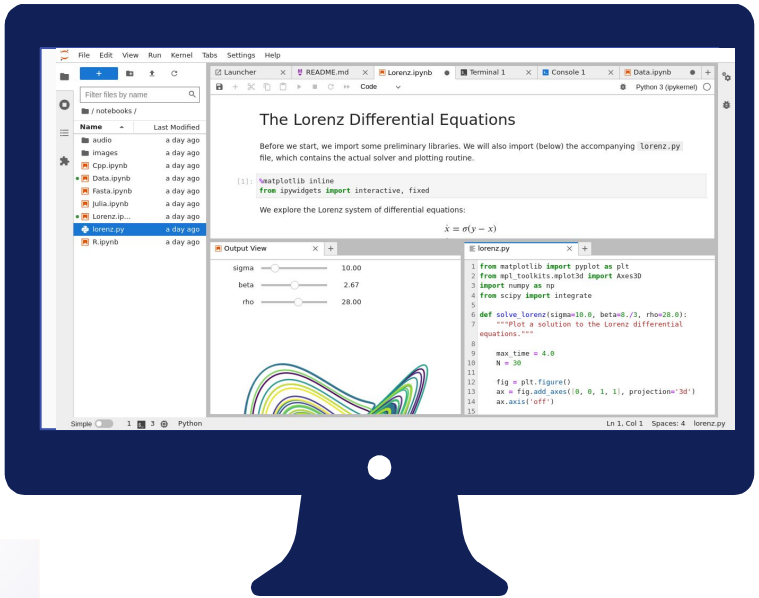
Aplicaciones

- Land cover mapping
- Computer security: malware and intrusion detection
- Face recognition
- Emotion recognition
- Fraud detection
- Financial decision-making
- Medicine, in neuroscience, proteomics and medical diagnosis

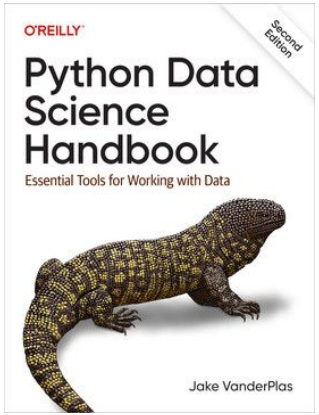
Resumen, al momento

- Un ensamble combina varios modelos en una predicción.
    - Funciona si cada uno es mejor que el azar y sus errores no coinciden.
- **Bayes**: el clasificador óptimo es un voto ponderado por $P(h|D)$; BMA elige, BMC combina.
- **Votación**: la dura cuenta votos; la suave promedia probabilidades y necesita que estén calibradas.
- **Stacking**: un metamodelo combina, entrenado con predicciones fuera del pliegue.
- **Bagging**: el mismo modelo en muestras *bootstrap*; baja la varianza. *Random forest* además sortea variables en cada nodo. OOB: validación gratis, salvo duplicados.
- En el vino, los ensambles de árboles arreglan al árbol, pero no superan con claridad a los mejores modelos de la unidad 4.

(Go to live notebook)



Extra Libro



GitHub

- 05.08-Random-Forests.ipynb

Gracias!

¿Alguna pregunta?

hussein@cicese.edu.mx



SCAN ME